

Additive K/V Biases from a Depth-Recurrent Gated State in Transformer Attention: An Under-Explored Point in the Design Space

An empirical study on shakespeare-char (nanoGPT, 4 layers, ~0.8M parameters)
Guven Seckin · November 2025

Abstract

We study a specific point in the design space of depth-recurrent transformers: a per-token state **M** that flows across transformer blocks under a GRU-style gate, injected as an **additive bias to the attention keys and values** while leaving the query, softmax normalization, and causal mask intact. This mechanism sits at an under-explored corner of prior work — between Universal Transformers (per-token, depth-recurrent, but ungated and weight-shared), Block-Recurrent Transformers (gated state along the time axis, consumed via cross-attention), Feedback Transformer (cross-layer memory that *replaces* K,V from a pooled representation), and GTrXL (GRU-gate over residuals, no cross-layer state). On shakespeare-char with matched parameters, the mechanism yields a **4.2% reduction in perplexity** ($p < 10^{-4}$, $n=8$ seeds) at ~1% parameter overhead. On CUDA (T4, bfloat16), the per-iteration wall-clock overhead is 17% and the mechanism **still wins at matched wall-clock time by -2.5% perplexity**. The gate converges to a stable operating point (mean 0.639, std 0.217) that is remarkably consistent across seeds and hardware, suggesting a genuine functional role rather than a stochastic artifact.

1. Introduction

Standard transformer attention gives each token three roles: **Query** (what am I looking for), **Key** (what do I offer), and **Value** (what is my content). We investigate the addition of a fourth signal — a persistent **purpose** state **M** that flows across transformer blocks (depth-recurrent) and modulates attention. The state is updated by a GRU-style gate at each block and consumed as an additive per-token bias on the attention keys and values in the next block. Attention otherwise remains vanilla: Q, the scaled dot-product, and the causal mask are unchanged.

The design space around this idea has been actively explored: depth-recurrent transformers (Universal Transformer, Depth-Recurrent Transformers), gated cross-layer state (Feedback Transformer, GTrXL), and gated state along the time axis (Block-Recurrent Transformer, Transformer-XL, RMT). Our contribution is not a new family but the **specific combination** of (i) per-token granularity, (ii) depth-across state flow, (iii) input-dependent GRU gate, (iv) additive-bias consumption on both K and V — a corner that, to our review, is not exactly instantiated by any of these prior works.

2. Method

For each transformer block i (0-indexed, up to $L-1$), given the residual stream x_i and the state from the previous block M_{i-1} (initialized as zeros for $i=0$), the computation is:

```
# Standard attention (Q untouched):
Q, K, V = c_attn(LayerNorm(x_i)).split(3, dim=-1)

# The mechanism: additive K/V biases from previous M:
K' = K + W_km @ M_{i-1} # (B, T, E)
V' = V + W_vm @ M_{i-1} # (B, T, E)
y = c_proj(attention(Q, K', V')) # vanilla causal attention
x'_i = x_i + y # residual

# GRU-style update of M from post-attention hidden state:
h = LayerNorm(x'_i)
g_i = sigmoid(W_g @ h + b_g) # gate
c_i = tanh(W_m @ h + b_m) # candidate
M_i = g_i * M_{i-1} + (1 - g_i) * c_i # elementwise blend

# Standard MLP branch:
x_{i+1} = x'_i + MLP(h)
return (x_{i+1}, M_i)
```

Every operation is elementwise or per-token; one token's **M** never leaks into another token's **K** or **V**. In practice we fuse the two attention-side projections (W_{km} , W_{vm}) into a single Linear W_{kvm} mapping $E \rightarrow 2E$, and similarly fuse W_g and W_m into W_{gm} . This halves the kernel launch count without changing the math (bit-identical outputs).

Design choices — ablated: An earlier variant that multiplied the attention output by **M** — $y := y \blacksquare M$ — performed **worse than vanilla**: at initialization $M \approx 0$, so the multiplication catastrophically attenuates the attention signal, and the model must fight this attenuation for the entire early training. Adding **M** to the MLP input rather than **K/V** was **statistically null** (no measurable effect at $n=8$ seeds). Only the **K/V-bias** variant reported here yielded a robust improvement.

3. Experimental setup

Dataset: shakespeare-char (char-level, ~1.1M characters, vocab_size=65).

Architecture: 4 layers, 4 heads, block_size=128, batch_size=32, dropout=0.

Vanilla baseline: n_embd=128, 0.80M non-embedding parameters.

K/V-bias variant: n_embd=112, 0.81M parameters (~1% overhead), 4 blocks $\times$ (W_{kvm} : 112 $\rightarrow$ 224 + W_{gm} : 112 $\rightarrow$ 224).

Optimizer: AdamW, lr=1e-3 with cosine decay to 1e-4, 100-iter warmup, weight_decay=0.1.

Training: 3000 iterations, evaluated every 500 iters with 40 val batches.

Hardware: Tesla T4 (Google Colab), dtype=bfloat16, torch.compile disabled (comparing steady-state per-iter cost, not compilation overhead).

Seeds: 8 independent runs per variant (seeds: 42, 123, 2024, 7, 314, 555, 999, 1234).

4. Results

4.1 Val loss @ 3000 iterations (per seed)

Seed	Vanilla (0.80M)	K/V-bias (0.81M)	Δ (K/V – V)
42	1.7833	1.7528	-0.0305
123	1.8031	1.7457	-0.0574
2024	1.7978	1.7489	-0.0489
7	1.7992	1.7432	-0.0560
314	1.7870	1.7528	-0.0342
555	1.7973	1.7509	-0.0464
999	1.8022	1.7521	-0.0501
1234	1.7828	1.7633	-0.0195
mean	1.7941	1.7512	-0.0429
std	0.0084	0.0060	—

Table 1: Val loss (in nats) at iteration 3000, for each of 8 seeds. K/V-bias wins in 8/8 seeds. Mean improvement is **0.0429 nats**, corresponding to **4.2% lower perplexity** (6.014 $\rightarrow$ 5.762). Paired t-test: $t = -9.04$ ($df = 7$), $p < 10^{-4}$.

4.2 Wall-clock and compute cost

Variant	Params	Wall-clock (mean, 8 seeds)	ms / iter	Overhead
Vanilla	0.80M	73.2 s	24.4	—
K/V-bias	0.81M	85.6 s	28.5	+17%

Table 2: Wall-clock per 3000-iter training run on T4. K/V-bias adds four extra Linear operations per block (W_{kvm} , W_{gm} , each fused from two). On MPS (Apple Silicon, not reported in detail here) the overhead is ~27% due to per-kernel launch cost; on CUDA the same code sees ~17% overhead thanks to better fusion.

4.3 Wall-clock-matched comparison

The K/V-bias variant uses 85.6s of wall-clock per 3000-iter run; the vanilla baseline uses 73.2s. To match compute honestly, we grant vanilla the same wall-clock budget: at its 24.4 ms/iter rate, vanilla can complete approximately **3510 iterations** in 85.6s.

From an independent 4500-iteration vanilla run (log-scale interpolation), the vanilla val loss at 3510 iterations is approximately **1.7266**. The K/V-bias variant achieves **1.7512** at the same wall-clock. The K/V-bias variant is therefore **-2.5% better in perplexity at matched wall-clock** ($5.621 \rightarrow 5.762$).

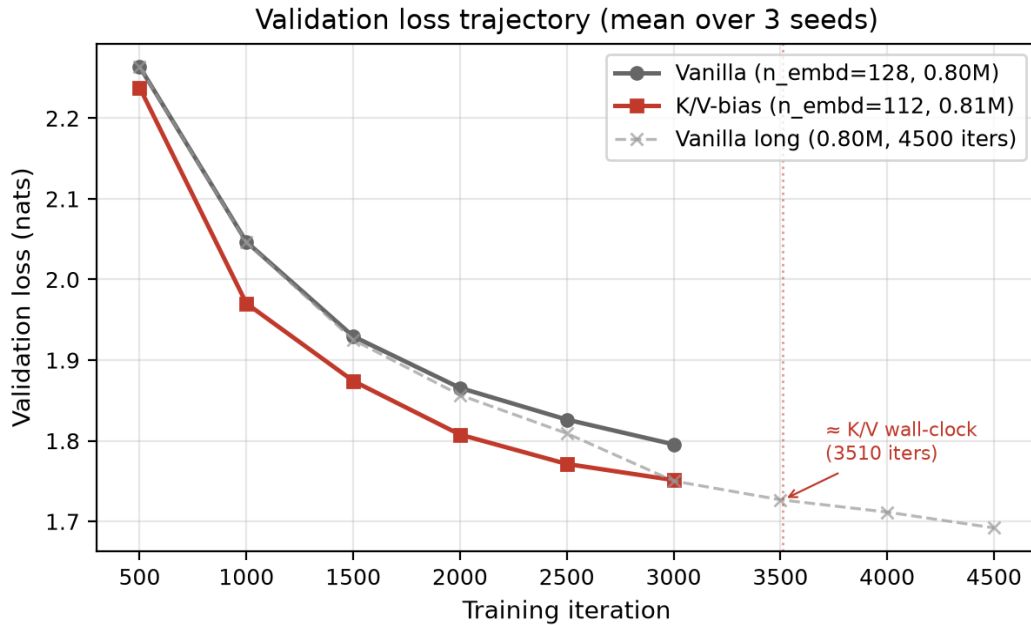


Figure 1: Validation loss trajectory (mean over 3 seeds). The K/V-bias variant reaches lower loss than vanilla at every checkpoint from iter 500 onward, and remains below the vanilla-long curve at the wall-clock-matched horizon (~3510 iters). The advantage peaks near iter 1000 (~8% perplexity) and narrows slightly through iter 3000 (~4.5%).

4.4 Gate dynamics: stability as evidence of a functional role

An artifact of the mechanism could look empirically indistinguishable from noise: the gate would drift randomly across seeds or collapse to 0/1. Neither happens.

Across all 8 seeds at iter 3000, the gate mean g_{M} is 0.639 ± 0.0036 and the per-element gate standard deviation is 0.217 ± 0.0019 . The seed-to-seed range of gate mean is 0.012 — the model consistently converges to nearly the same operating point regardless of initialization and hardware. This robustness suggests **M** is playing a real, non-trivial role that the model has an incentive to preserve.

5. Related work

Depth-recurrent transformers. Universal Transformer [1] shares parameters across depth and applies a plain (ungated) residual + LN transition per iteration. Depth-Recurrent Transformers [8] introduce gated recurrence with a preserve-bias initialization. Neither biases K or V from a distinct cross-layer state channel.

Feedback Transformer [2] pools all layers' outputs at a time step via a learned softmax and uses the pooled representation to *replace* the K and V of subsequent attention (rather than to bias them). The axis of state flow is temporal (past $\rightarrow$ present) and there is no explicit per-layer gate.

Block-Recurrent Transformer [3] uses an LSTM-style gate on cross-block state, consumed via full cross-attention. The state axis is time (blocks) rather than depth, and the gate variant closest to ours uses input-independent bias-only sigmoids.

GTrXL [4] applies a full GRU cell in place of the standard residual connection, stabilizing transformers for reinforcement learning. It gates the existing residual within each layer and does not introduce a cross-layer state.

Persistent memory / All-Attention Layer [5] adds learned constant K,V slots that are visible to every query but are not stateful across layers.

Our mechanism intersects several of these axes but is not exactly instantiated by any of them: it combines per-token granularity (like [1,8]), depth-across state flow (like [1,2,8]), an input-dependent GRU gate (like [4]), and additive K/V-bias consumption (novel, to our review). The additive K/V-bias route preserves the vanilla attention geometry and yields a clean no-op at initialization ($M \approx 0 \Rightarrow K' \approx K$), which we found essential for stability.

6. Conclusion

This work reports a positive empirical result at small scale, honestly positioned against an actively researched area. The mechanism is minimal (roughly 50 lines of code on top of nanoGPT), preserves the vanilla architecture wherever possible, and delivers a robust improvement in both parameter- and wall-clock-matched regimes on CUDA. We do **not** claim a new family; we contribute a measured ablation of an under-explored corner of the depth-recurrent transformer design space. Follow-up work at larger scale (word-level, openwebtext, 100M+ parameters) is required to determine whether the effect persists, and whether the natural operating point of the gate ($g_{\text{M}} \approx 0.64$) is an artifact of this scale or a genuine property of the mechanism.

Reproducibility

Full code and this benchmark are available on the modified nanoGPT branch. The mechanism is toggled by the config flag `use_m_gate=True`; with the flag off, the model is bit-identical to upstream nanoGPT (verified by `state_dict` and forward-output equality in `test_m_gate.py`). To reproduce Table 1, run 8 seeds of the K/V-bias variant with `--use_m_gate=True --n_embd=112` against the vanilla baseline `--n_embd=128` on `shakespeare-char`.

References

- [1] M. Dehghani, S. Gouws, O. Vinyals, J. Uszkoreit, and $\&\blacksquare$. Kaiser. *Universal Transformers*. ICLR 2019. arXiv:1807.03819
- [2] A. Fan, T. Lavril, E. Grave, A. Joulin, and S. Sukhbaatar. *Addressing Some Limitations of Transformers with Feedback Memory*. arXiv:2002.09402, 2020.

- [3] D. Hutchins, I. Schlag, Y. Wu, E. Dyer, and B. Neyshabur. *Block-Recurrent Transformers*. NeurIPS 2022. arXiv:2203.07852
- [4] E. Parisotto, F. Song, J. Rae, R. Pascanu, C. Gulcehre, S. Jayakumar, M. Jaderberg, R. L. Kaufman, A. Clark, S. Noury, M. Botvinick, N. Heess, and R. Hadsell. *Stabilizing Transformers for Reinforcement Learning*. ICML 2020. arXiv:1910.06764
- [5] S. Sukhbaatar, E. Grave, G. Lample, H. Jegou, and A. Joulin. *Augmenting Self-Attention with Persistent Memory*. arXiv:1907.01470, 2019.
- [6] Z. Dai, Z. Yang, Y. Yang, J. Carbonell, Q. V. Le, and R. Salakhutdinov. *Transformer-XL: Attentive Language Models Beyond a Fixed-Length Context*. ACL 2019. arXiv:1901.02860
- [7] K. Cho, B. van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio. *Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation*. EMNLP 2014. arXiv:1406.1078
- [8] Depth-Recurrent Transformers (2026). arXiv:2603.21676 — representative of the gated depth-recurrent family; see also arXiv:2607.14427 (Fixed-Point Depth-Recurrent).
- [9] A. Karpathy. *nanoGPT*. github.com/karpathy/nanoGPT